

Salesportal — Complete Project Documentation

- Salesportal — Complete Project Documentation
 - 1. Product overview
 - 2. System architecture
 - 3. Technology stack & dependencies
 - 4. Repository layout
 - 5. Environment & local setup
 - 6. Authentication & security model
 - 7. Backend API reference
 - 8. Database reference
 - 9. Frontend reference
 - 10. Core business workflows
 - 11. Validation rules
 - 12. Error-handling & coding conventions
 - 13. Operations & maintenance scripts

Salesportal — Complete Project Documentation

Professional, end-to-end documentation for the Salesportal (“Recruweb”) platform, generated from the live codebase.

1. Product overview

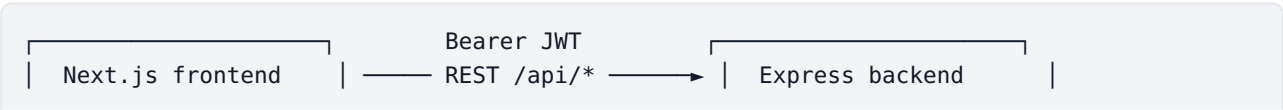
Salesportal is a full-stack B2B field-sales platform connecting three user roles through dedicated portals:

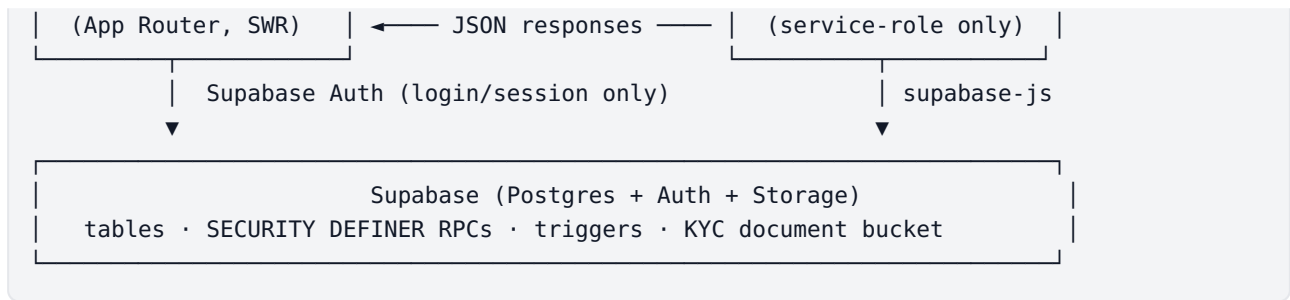
Role	Portal	Purpose
Admin	/admin	Platform operations: catalog moderation, field-sale verification, KYC review, commission ledger & payout approval, refunds, analytics, support inbox
Field Sales Officer (FSO)	/field	Independent sales agents — registerable from anywhere in the world — who sell client products, track earnings/commissions, run a personal CRM, and complete KYC
Client	/client	Business customers who list products, receive orders, manage inventory, handle returns, and monitor field-sales performance on their catalog

Key product capabilities:

- Global FSO registration with international phone numbers (36 country dial codes).
- Commission engine: approved field sales automatically generate commissions, synced to per-officer wallets; officers request payouts, admins approve.
- Two-sided KYC (officers and clients) with document upload to Supabase Storage, signed URLs, and an admin review queue.
- Built-in CRM for officers: leads, follow-ups, tasks, meetings, notes.
- Returns & refunds lifecycle spanning client, admin, and order records.
- Real-time-feeling dashboards backed by SQL RPCs (single round-trip aggregation).
- In-app notifications with per-user settings and web-push subscription support.
- Support chat: one thread per client/field user, admin inbox on the other side.

2. System architecture





Principles:

- The browser **never** touches data tables directly. Supabase is used client-side for authentication only; every data operation flows through the Express API.
- The Supabase **service-role key** lives exclusively in `backend/.env`. Because it bypasses RLS, every controller scopes queries by the `verified req.user.id`.
- Heavy aggregation is pushed into `SECURITY DEFINER SQL` functions so dashboards cost one round trip; controllers keep query-level fallbacks where schema drift is possible.
- Same-origin frontend calls are proxied to the backend through the `/backend` rewrite in `next.config.mjs` (overridable with `NEXT_PUBLIC_API_URL`).

3. Technology stack & dependencies

Backend (`backend/package.json`)

Category	Packages
Core	<code>express 5</code> , <code>dotenv</code> , <code>morgan (logging)</code> , <code>compression</code>
Security	<code>helmet</code> , <code>cors</code> , <code>express-rate-limit</code> , <code>jsonwebtoken</code> , <code>bcrypt</code> , <code>cookie-parser</code>
Data	<code>@supabase/supabase-js 2</code> , <code>@supabase/ssr</code> , <code>pg</code>
Uploads/media	<code>multer</code> , <code>cloudinary</code>
Messaging	<code>nodemailer</code> , <code>web-push</code> , <code>socket.io</code>
Caching (available)	<code>ioredis</code> , <code>redis</code>

Scripts: `npm run dev (node --watch index.js)`, `npm start`.

Frontend (`frontend/package.json`)

Category	Packages
Framework	<code>next 16.2</code> , <code>react 19</code> , <code>react-dom 19</code>
Data	<code>swr 2</code> , <code>@supabase/supabase-js</code> , <code>@supabase/ssr (auth only)</code>
UI	<code>Tailwind CSS 4</code> , <code>shadcn</code> , <code>@base-ui/react</code> , <code>lucide-react</code> , <code>react-icons</code> , <code>framer-motion</code> , <code>tw-animate-css</code>
Utilities	<code>clsx</code> , <code>tailwind-merge</code> , <code>class-variance-authority</code> , <code>@vercel/analytics</code>

Scripts: `next dev`, `next build`, `next start`, `eslint .`

4. Repository layout

```
Salesportal/
├── backend/
│   ├── index.js           Entry point (.env load, server boot)
│   ├── package.json
│   ├── db/               SQL schemas, migrations, seed & repair scripts
│   └── src/
│       └── app.js         helmet · CORS · rate limits · mounts · 404/error
├── handlers
│   ├── config/supabase.js supabaseAdmin (service-role client)
│   ├── middleware/auth.js requireAuth · requireRole · token cache
│   ├── controllers/      13 active domain controllers
│   ├── routes/           11 routers (one per portal/domain)
│   └── utils/            phone.js · regions.js · notifications.js
├── frontend/
│   ├── app/              45 route files across 4 portals (see §9)
│   └── components/       ui/ · forms/ · kyc/ · chat/ · notifications/ ·
├── portal/ · landing/
│   └── lib/              api/ clients · validation/ · supabase/ (auth) ·
├── store/ · types
│   ├── next.config.mjs   /backend → API rewrite
│   └── package.json
├── docs/                This documentation (+ .docx / .pdf builds)
├── REFUNDS_QUICKSTART.md
└── RETURNS_REFUNDS_IMPLEMENTATION.md
```

5. Environment & local setup

Backend `.env`

Variable	Purpose
<code>SUPABASE_URL</code>	Supabase project URL
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Service-role key — server only, bypasses RLS
<code>ADMIN_EMAIL</code>	The ONLY email honored as admin (super-admin lockdown)
<code>PORT</code>	API port (default 4000)

Frontend `.env.local`

Variable	Purpose
<code>NEXT_PUBLIC_SUPABASE_URL</code> / <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Supabase Auth (login/session only)
<code>NEXT_PUBLIC_API_URL</code>	Backend origin when not using the <code>/backend</code> rewrite

Run

```
cd backend && npm install && npm run dev      # API on :4000
cd frontend && npm install && npm run dev      # Next.js on :3000
```

Database bootstrap (idempotent, in order)

1. `backend/db/schema.sql` — categories, products, orders
2. `backend/db/portal-schema.sql` — profiles, commissions, dashboard RPCs, triggers
3. Domain schemas: `kyc-schema.sql`, `sales-submissions-schema.sql`, `crm-schema.sql`, `field-schema.sql`, `chat-schema.sql`, `notifications-schema.sql`, `commission-wallet-schema.sql`
4. `performance-indexes.sql`
5. Seeds: `node db/seed.js`, `node db/seed-portal.js`, `node db/create-admin.js`

6. Authentication & security model

- **Identity:** Supabase Auth (email + password). Sign-in happens in the browser against Supabase; sign-up goes through `POST /api/auth/signup`, which uses the backend Admin API to create auto-confirmed accounts with a role in metadata.
- **Session:** the Supabase access token is sent as `Authorization: Bearer <jwt>` on every API call.
- **`requireAuth`** (`src/middleware/auth.js`) verifies the token and attaches `{ id, email, role, fullName }` to `req.user`. Hardening:
 - Bounded in-memory verification cache — SHA-256 token hashes, TTL = min(JWT expiry, 5 minutes), FIFO eviction at 1,000 entries — one upstream verification per token rather than per request.
 - 8-second hard timeout on verification; a hung upstream returns a retryable 503.
- **Super-admin lockdown:** `role=admin` is honored only when the token's email equals `ADMIN_EMAIL`; any other token claiming admin is silently demoted to `client` before role checks run.
- **`requireRole(...roles)`** returns 403 unless `req.user.role` matches.
- **Scoping:** client/field endpoints always filter by `req.user.id` from the verified token — never from params or body (critical because the service-role client bypasses RLS).
- **Transport hardening in `app.js`:** helmet, CORS allow-list, JSON body capped at 1 MB, global rate limit on `/api`, stricter limit on `/api/auth`, JSON 404, and a final error handler that never leaks stack traces.
- **Passwords:** hashed by Supabase Auth (bcrypt) — never stored or logged by the application.

7. Backend API reference

All routes below are prefixed as mounted in `src/app.js`. Auth column shows the required role; “any” = any authenticated user.

7.1 `/api/auth` — public (strict rate limit)

Method	Path	Auth	Purpose
POST	<code>/api/auth/signup</code>	public	Create account (fullName, email, password, role) via Supabase Admin API

7.2 `/api/catalog`

Method	Path	Auth	Purpose
GET	<code>/categories</code>	public	Category list
GET	<code>/products</code>	client	List own products
POST	<code>/products</code>	client	Create product (enters admin review)
PATCH	<code>/products/:id</code>	client	Update own product
DELETE	<code>/products/:id</code>	client	Remove own product

7.3 `/api/orders`

Method	Path	Auth	Purpose
GET	<code>/</code>	any	Orders, role-scoped (client sees own; admin sees all)
GET	<code>/summary</code>	any	KPI rollup (orders_summary RPC)
PATCH	<code>/:id</code>	any	Status transition (role-validated)

7.4 `/api` — returns & refunds

Method	Path	Auth	Purpose
GET	<code>/returns · /returns/:id</code>	any	Return requests, role-scoped
POST	<code>/returns</code>	client	Open a return against an order
PATCH	<code>/returns/:id</code>	admin	Approve / reject / progress a return
GET	<code>/refunds · /refunds/:id</code>	any	Refund visibility, role-scoped
POST	<code>/refunds</code>	admin	Issue a refund
PATCH	<code>/refunds/:id/status</code>	admin	Progress refund (e.g. processing → settled)

7.5 /api/chat

Method	Path	Auth	Purpose
GET	/thread · /thread/ summary	client/field	Own support thread + unread summary
POST	/messages	client/field	Send message (bumps thread via chat_bump_thread)
GET	/threads · / threads/:id	admin	Inbox list / one conversation
POST	/threads/:id/ messages	admin	Reply

7.6 /api/field — role: field

Method	Path	Purpose
GET/PUT	/profile	Officer profile; PUT validates the international phone server-side
GET	/products	Sellable (approved) catalog
GET/POST	/orders	List / place field orders (place_field_order RPC — atomic order + commission)
GET	/summary	“My Day” KPIs — officer_summary RPC with an in-controller fallback that recomputes from orders/ commissions/profiles if the RPC is missing or drifted; optional visits/ leads counts degrade to 0
GET	/performance	Daily/monthly trends (perf RPCs)
GET	/leaderboard	Region + national rankings
GET	/commissions	Commission ledger
GET	/wallet · POST /wallet/payout	Wallet balances / payout request
GET/POST	/sales	Sale submissions (customer phone validated)
GET	/kyc/me	KYC state + documents
POST	/kyc/documents · DELETE /kyc/ documents/:id	Upload / remove docs
PATCH	/kyc/submit	Move draft → pending
GET	/kyc/documents/:id/url	Short-lived signed URL

7.7 `/api/field/crm` — role: field (all rows scoped to the officer)

Area	Endpoints
Leads	GET/POST <code>/leads</code> , GET/PATCH/DELETE <code>/leads/:id</code>
Follow-ups	GET/POST <code>/leads/:id/follow-ups</code> , PATCH/DELETE <code>/follow-ups/:id</code> , GET <code>/follow-ups</code>
Tasks	GET/POST <code>/tasks</code> , PATCH/DELETE <code>/tasks/:id</code>
Meetings	GET/POST <code>/meetings</code> , PATCH/DELETE <code>/meetings/:id</code>
Notes	GET/POST <code>/leads/:id/notes</code> , DELETE <code>/notes/:id</code>

7.8 `/api/client` — role: client

Method	Path	Purpose
GET	<code>/overview</code>	Client dashboard (<code>client_overview</code> RPC family)
—	<code>/kyc/*</code>	Same 5-endpoint KYC set as field (shared <code>kyc.controller</code>)

7.9 `/api/admin` — role: admin

Method	Path	Purpose
GET	<code>/products</code> · <code>/products/counts</code>	Moderation queue + counts by status
PATCH	<code>/products/:id/review</code>	Approve / reject a product
GET	<code>/sales</code> · PATCH <code>/sales/:id/review</code>	Field-sale verification (approval creates the commission)
GET	<code>/kpis</code> · <code>/overview</code>	Platform dashboards (<code>admin_summary</code> , <code>region/revenue/top-product</code> RPCs)
GET	<code>/clients-overview</code> · <code>/officers-overview</code>	Account overviews
GET	<code>/commissions</code> · <code>/commissions/summary</code>	Ledger + rollup
PATCH	<code>/commissions/:id/status</code>	pending → settled
GET	<code>/commissions/payouts</code> · PATCH <code>/commissions/payouts/:id/review</code>	Payout queue / approve-reject
GET	<code>/kyc</code> · <code>/kyc/:id</code> · PATCH <code>/kyc/:id/review</code>	KYC review queue, detail, decision

7.10 Reports — `/api/admin/reports` and `/api/field/reports`

Method	Path	Auth	Purpose
GET	<code>/api/admin/reports/summary</code>	admin	Revenue, orders, officers, clients KPIs
GET	<code>/api/admin/reports/sales-trend?days</code>	admin	Time-series
GET	<code>/api/admin/reports/top-officers?limit</code>	admin	Ranking
GET	<code>/api/admin/reports/kyc-stats</code>	admin	KYC funnel
GET	<code>/api/admin/reports/commission-summary</code>	admin	Commission rollup
GET	<code>/api/field/reports?days</code>	field	Officer personal analytics (trend, earnings, rank)

7.11 Notifications — `/api/notifications` (+ `/api/admin/notifications`)

Method	Path	Purpose
GET	<code>/ · /unread-count</code>	Feed + badge
PATCH	<code>/:id/read · /read-all</code>	Mark read
DELETE	<code>/:id · /</code>	Remove one / clear all
GET/PUT	<code>/settings</code>	Per-user notification preferences
POST	<code>/push-subscription</code>	Register web-push subscription

7.12 Controllers

Active: `auth`, `catalog`, `orders`, `returns`, `refunds`, `chat`, `field`, `sales`, `commission`, `crm`, `kyc`, `overview` (admin+client dashboards), `reports`, `notifications`. (`client.controller.js` and `profile.controller.js` are empty placeholders — their duties live in `overview` and `kyc/field.`)

8. Database reference

8.1 Schema files (apply order in §5)

`schema.sql`, `portal-schema.sql`, `kyc-schema.sql`, `sales-submissions-schema.sql`, `crm-schema.sql`, `field-schema.sql`, `chat-schema.sql`, `notifications-schema.sql`, `commission-wallet-schema.sql`, `performance-indexes.sql`, `patch-client-id-nullable.sql`.

8.2 Tables (22)

Table	Purpose / key columns
<code>categories</code>	Product taxonomy

Table	Purpose / key columns
products	Catalog; client_id owner, admin review status , price, imagery
orders	amount , qty , status , channel (field/client), officer_id , placed_at
profiles	Keyed by user_id ; full_name , phone (canonical +<dial><10 digits>), region , city , monthly_target
commissions	officer_id , amount , status (pending / settled), created_at
officer_wallets	Derived balances; maintained by sync_wallet_on_commission trigger
payout_requests	Officer withdrawal requests; admin review status
sales_submissions	Field sale entries: officer_id , officer_name snapshot , total_amount , status (pending / approved / rejected), customer_phone , created_at — Δ NOT sale_amount / submitted_at
kyc_submissions	One per user; status draft-pending-approved/rejected , user_role , submitted_at
kyc_documents	Storage refs; served via short-lived signed URLs
returns / refunds	Return requests and admin-issued refunds with status lifecycles
leads , lead_follow_ups , lead_notes	CRM, scoped by assigned_officer_id
crm_tasks , meetings	Officer to-dos and calendar
visits	Field visit log (officer_id , visited_at) — optional for dashboards
chat_threads , chat_messages	Support chat
notifications , notification_settings	Feed + per-user prefs

8.3 RPC functions & triggers (SECURITY DEFINER)

Function	Consumed by
officer_summary(p_officer_id)	GET /api/field/summary (with controller fallback)
officer_perf_daily / officer_perf_monthly	Field performance
officer_leaderboard	Rankings
place_field_order(...)	Atomic field order + commission insert
admin_summary , admin_region_stats , admin_revenue_by_month , admin_top_products	Admin dashboards
admin_clients_overview , admin_officers_overview	Admin account pages
client_overview family	Client dashboard
orders_summary	Orders KPI endpoint

Function	Consumed by
<code>admin_user_ids</code> , <code>user_names</code>	Safe identity lookups without exposing the auth schema
<code>chat_bump_thread</code>	Thread recency on new message
<code>sync_wallet_on_commission</code> (trigger)	Wallet consistency
<code>touch_updated_at</code> , <code>update_kyc_submission_updated_at</code> (triggers)	<code>updated_at</code> maintenance

9. Frontend reference

9.1 Route map (45 files in `app/`)

Public / auth

Route	Page
<code>/</code>	Landing (marketing + portal entry)
<code>/auth/login</code> , <code>/auth/sign-up</code> , <code>/auth/sign-up-success</code> , <code>/auth/error</code>	Supabase Auth flows
<code>/field/join</code>	Public global FSO registration (country-code phone input)

Admin portal `/admin` (11 pages, wrapped by `admin/layout.tsx` shell)

`/admin` dashboard · `/admin/products` moderation · `/admin/verifications` field-sale review · `/admin/kyc` review queue · `/admin/commissions` ledger + payouts · `/admin/clients` · `/admin/officers` · `/admin/reports` analytics · `/admin/inbox` support chat · `/admin/notifications`.

Field portal `/field` (15 pages)

`/field` My Day dashboard · `/field/products` · `/field/sell` sale submission · `/field/orders` · `/field/earnings` wallet + payouts · `/field/performance` · `/field/reports` · `/field/crm` (+ `/follow-ups`, `/tasks`, `/meetings`) · `/field/leads` · `/field/kyc` · `/field/profile` · `/field/notifications`.

Client portal `/client` (10 pages)

`/client` overview · `/client/products` · `/client/inventory` · `/client/launch` product launch · `/client/orders` · `/client/returns` · `/client/performance` · `/client/kyc` · `/client/profile` · `/client/notifications`.

9.2 Shared components

Directory	Contents
<code>components/ui/</code>	shadcn/ui primitives (button, input, card, dialog, table, ...)
<code>components/forms/</code>	<code>phone-input.tsx</code> — country dropdown (36 countries) + 10-digit national field, digit-only sanitizing on type and paste, aria-wired errors

Directory	Contents
<code>components/kyc/</code>	<code>kyc-upload.tsx</code> shared by field & client KYC
<code>components/chat/</code>	User chat widget + admin inbox pieces
<code>components/notifications/</code>	Bell, feed, settings panel
<code>components/portal/</code> , <code>landing/</code> , <code>launch/</code> , <code>auth/</code> , <code>client/</code>	Shells and page sections

9.3 `lib/` modules

Path	Purpose
<code>lib/api/*.ts</code>	Typed API client per domain (<code>field</code> , <code>admin</code> , <code>reports</code> , <code>overview</code> , <code>kyc</code> , <code>crm</code> , <code>chat</code> , <code>notifications</code> , <code>sales</code> , <code>commissions</code> , <code>client</code> , <code>auth</code>) — wraps <code>fetch</code> with the Bearer token; consumed via SWR
<code>lib/validation/phone.ts</code>	Phone source of truth: <code>COUNTRIES</code> (36), <code>validatePhoneIntl</code> , <code>splitPhone</code> , <code>sanitizeNationalInput</code>
<code>lib/supabase/</code>	Browser/server Supabase clients — auth only
<code>lib/auth/roles.ts</code>	<code>PortalRole</code> union + helpers
<code>lib/store/</code>	Client stores (e.g. <code>vendor-store.tsx</code>)
<code>lib/types.ts</code> , <code>lib/utils.ts</code>	Shared types, <code>formatINR</code> , <code>cn</code>

9.4 Data-fetching pattern

SWR keyed by endpoint; every data page implements loading skeletons, a retryable error state, and a real empty state — no mock data anywhere. Percentage/growth math is divide-by-zero safe. Dashboards prefer one RPC-backed endpoint; partial back-end failures render a non-blocking warning banner instead of a blank screen.

10. Core business workflows

10.1 Field sale → commission → payout

1. Officer submits a sale (`POST /api/field/sales`) — validated (product, quantity, customer phone), stored in `sales_submissions` as pending.
2. Admin reviews at `/admin/verifications` (`PATCH /api/admin/sales/:id/review`).
3. **Approval** creates a `commissions` row; the `sync_wallet_on_commission` trigger updates `officer_wallets`.
4. Officer sees earnings at `/field/earnings` and requests a payout (`POST /api/field/wallet/payout → payout_requests`).
5. Admin approves/rejects at `/admin/commissions` (`PATCH /api/admin/commissions/payouts/:id/review`).

10.2 KYC lifecycle (officer & client)

draft (upload documents) → PATCH /kyc/submit → pending → admin decision (PATCH /api/admin/kyc/:id/review) → approved or rejected (rejection includes a reason; the user can fix documents and resubmit). Documents live in Supabase Storage and are only ever served through short-lived signed URLs.

10.3 Product listing

Client creates a product (POST /api/catalog/products) → admin moderation at /admin/products → approved products become sellable in /field/products .

10.4 Returns & refunds

Client opens a return against an order → admin progresses it (PATCH /returns/:id) → admin issues a refund (POST /refunds) → refund is settled (PATCH /refunds/:id/status). Consistency between orders, returns and refunds is repairable via db/fix-return-refund-consistency.js .

10.5 Support chat

Client/field users own exactly one thread (auto-created on first message); messages POST /api/chat/messages bump the thread; admins work the inbox at /admin/inbox .

11. Validation rules

Phone numbers (global)

- UI: country dropdown (India, Nepal, UK, US + 32 more) + exactly-10-digit national field; non-digits stripped on type and paste; an 11-digit paste starting with 0 loses the trunk zero.
- Canonical stored format: +<dia><10 digits> (e.g. +919822014455 , +9779812345678).
- Server re-validates with backend/src/utls/phone.js (normalizePhone) — identical rules; legacy bare/0-prefixed Indian numbers are accepted and canonicalized under +91 .
- Editing a saved profile round-trips through splitPhone so dropdown and field re-populate, including legacy rows.

Other

- Money: numeric columns; NaN-safe parsing; 2-decimal rounding.
- Order/sale quantities: positive integers, bounded.
- Emails: normalized to lowercase before Supabase Auth calls.
- All list endpoints paginate or cap result sizes (defensive limit).

12. Error-handling & coding conventions

- Controllers log details server-side — console.error("[domain:fn]", err.message) — and return only generic messages to clients (no stack traces, no SQL).
- **Core** queries fail loudly (HTTP 500); **enrichment** queries degrade gracefully (zeros / fallback labels) and are logged.

- Optional tables (e.g. `visits`, `leads` in the field-summary fallback) never fail a whole endpoint.
- Column-name contract: `sales_submissions.total_amount + created_at` (never `sale_amount / submitted_at` — the historical cause of Reports-page breakage).
- Officer display names: `profiles.full_name` first, then the `sales_submissions.officer_name` snapshot — never render “Unknown”.
- Routers stay thin; one controller per domain; shared logic lives in `utils/`.

13. Operations & maintenance scripts

Run as `node backend/db/<script>.js` with backend `.env` present.

Script	Purpose
<code>create-admin.js</code>	Provision the single super-admin (must match <code>ADMIN_EMAIL</code>)
<code>seed.js</code> / <code>seed-portal.js</code>	Base catalog + portal demo data
<code>seed-refunds.js</code> , <code>seed-client-returns.js</code> , <code>migrate-and-seed-refunds.js</code>	Returns/refunds data
<code>migrate-portal.js</code> , <code>migrate-field.js</code> , <code>migrate-chat.js</code>	Apply schema files programmatically
<code>enrich-catalog.js</code> , <code>taxonomy.js</code> , <code>update-real-products.js</code>	Catalog enrichment
<code>refresh-product-images.js</code> , <code>update-product-images.js</code> , <code>fix-duplicate-images.js</code>	Product imagery maintenance
<code>fix-return-refund-consistency.js</code>	Repair return/refund pairs

End of documentation. Regenerate the `.docx` and `.pdf` builds from this file after any update (`pandoc --toc`; see `docs/` folder).